



DD

Design Document

ROAd – Autonomous Mobility Application

Authors

Margherita Bertipaglia
David Cazzaniga
Pasquale Ludovico Ignarra

University

Politecnico di Milano

Course

Software Engineering for Automation

Academic year

2025–2026

Date

July 2026

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Definitions, Acronyms, Abbreviations	2
1.3	Revision history	3
1.4	Document Structure	3
2	Architectural Design	4
2.1	Overview and architectural style	4
2.2	Component view	4
2.3	Class view	6
2.3.1	AllocationManager (Strategy)	6
2.3.2	Robotaxi (State)	7
2.3.3	NotificationManager (Observer)	7
2.4	Runtime view	8
2.5	Deployment view	10
2.6	Selected architectural styles and patterns	11
2.6.1	Strategy — allocation algorithms	11
2.6.2	Observer — state-change notifications	11
2.6.3	State — robotaxi lifecycle	11
2.6.4	Other patterns and styles	12
3	User Interface Design	13
3.1	Passenger mobile app	13
3.2	Operator web dashboard	13
4	Requirements Traceability	15
4.1	Functional requirements	15
4.2	Non-functional requirements	16
5	Implementation, Integration and Test Plan	17
5.1	Feature prioritisation	17
5.2	Implementation and integration order	17
5.3	Testing strategy	18
6	References	19

Introduction

1.1 Purpose

This is the Design Document (DD) of ROAd (Robotaxi Optimized Allocation). The document describes *how* the system is built: the software architecture, the components and their interfaces, the way they interact at runtime, the design patterns we adopt, the user interface, the mapping from requirements to design, and the plan to implement, integrate and test the system.

Main Goals

ROAd supports the management of an autonomous taxi fleet in a city. Passengers can request immediate rides or book rides in advance from a mobile app; the system assigns a suitable robotaxi according to a vehicle-allocation strategy that the fleet operator can choose at runtime. The system tracks the lifecycle of each robotaxi, notifies passengers about relevant events, lets the operator monitor the fleet from a web dashboard, and proactively rebalances available robotaxis toward areas of expected high demand. Autonomous driving, mapping and payment are treated as external dependencies and are not implemented as part of the system.

The design explicitly addresses these goals in terms of design patterns:

- ▶ **Strategy** pattern for switching between allocation algorithms.
- ▶ **Observer** pattern for notifying users when the state of a vehicle changes.
- ▶ **State** pattern for managing the lifecycle of a robotaxi.

1.2 Definitions, Acronyms, Abbreviations

Definitions

- ▶ **Component**: A modular, replaceable unit of the system that provides and requires services through well-defined interfaces.
- ▶ **Tier**: A physical (deployment) layer of the architecture.
ROAd is organised in three tiers: presentation, application, data.
- ▶ **S2B Software To Be**: The ROAd backend together with the two clients.
- ▶ **Manager**: An internal backend component responsible for one coherent area of the application logic (e.g. AllocationManager).
- ▶ **Gateway**: A component that mediates access to an external system, hiding its specific protocol from the rest of the system.

- ▶ **Context:** In the State/Strategy patterns, the object that delegates part of its behaviour to a separate state/strategy object.

Acronyms

- ▶ DD (Design Document)
- ▶ RASD (Requirements Analysis and Specification Document)
- ▶ API (Application Programming Interface)
- ▶ REST (Representational State Transfer)
- ▶ MVC (Model–View–Controller)
- ▶ ETA (Estimated Time of Arrival)
- ▶ FSM (Finite State Machine)
- ▶ UML (Unified Modeling Language)

Abbreviations

- ▶ [R_n] n-th functional requirement
- ▶ [NFR_n] n-th non-functional requirement
- ▶ [C_n] n-th component
- ▶ [UC_n] n-th use case

1.3 Revision history

Version	Date	Description
1.0	July 11, 2026	Initial release of the Design Document

1.4 Document Structure

- ▶ **Chapter 1 Introduction:** Purpose, terminology, revision history, structure.
- ▶ **Chapter 2 Architectural Design:** The architectural style, the component view, the class view of the most relevant components, the runtime view, the deployment view, and the selected styles and design patterns.
- ▶ **Chapter 3 User Interface Design:** An overview of the passenger app and of the operator dashboard through wireframes.
- ▶ **Chapter 4 Requirements Traceability:** How the requirements of the RASD are satisfied by the components and patterns of this document.
- ▶ **Chapter 5 Implementation, Integration and Test Plan:** The order in which we plan to implement, integrate and test the subcomponents.
- ▶ **Chapter 6 References**

Architectural Design

2.1 Overview and architectural style

ROAd adopts a **three-tier client-server** architecture:

- ▶ A **presentation tier** with two thin clients:
 1. The Passenger Mobile App
 2. The Operator Web Dashboard
- ▶ An **application tier** for the ROAd backend (the S2B core), which contains all the management logic, organised as a set of cooperating components behind an API Gateway.
- ▶ A **data tier** for a single relational database that stores accounts, requests, rides, vehicles and reservations.

This style is useful for three main reasons.

First, the two kinds of user need different clients (a mobile app for passengers, a data-rich web console for operators), but they must work on the same, consistent state: a centralized server is the natural place to realize this constraint, which is also what lets us prevent conflicting assignments (NFR4, C1).

Second, the application tier is internally *layered*: the business logic never talks to an external system directly but always through the `ExternalServicesGateway`, so the volatile parts (mapping provider, demand source, vehicle protocol) are isolated from the stable core (NFR8, separation of concerns). Third, the authoritative shared state is stored in the data tier, while the application tier keeps only disposable local caches. The backend can therefore be replicated horizontally, which supports scalability (NFR3) and concurrency (NFR1).

Inside the backend we also use, locally, two well-known styles: **MVC** on the client side (the views are a function of an observable model) and an **event-driven** mechanism for notifications, realised with the Observer pattern (Section 2.6).

2.2 Component view

Figure 2.1 shows the high-level component diagram. The backend is decomposed into single-responsibility managers; each one exposes a small interface and depends on the others only through those interfaces.

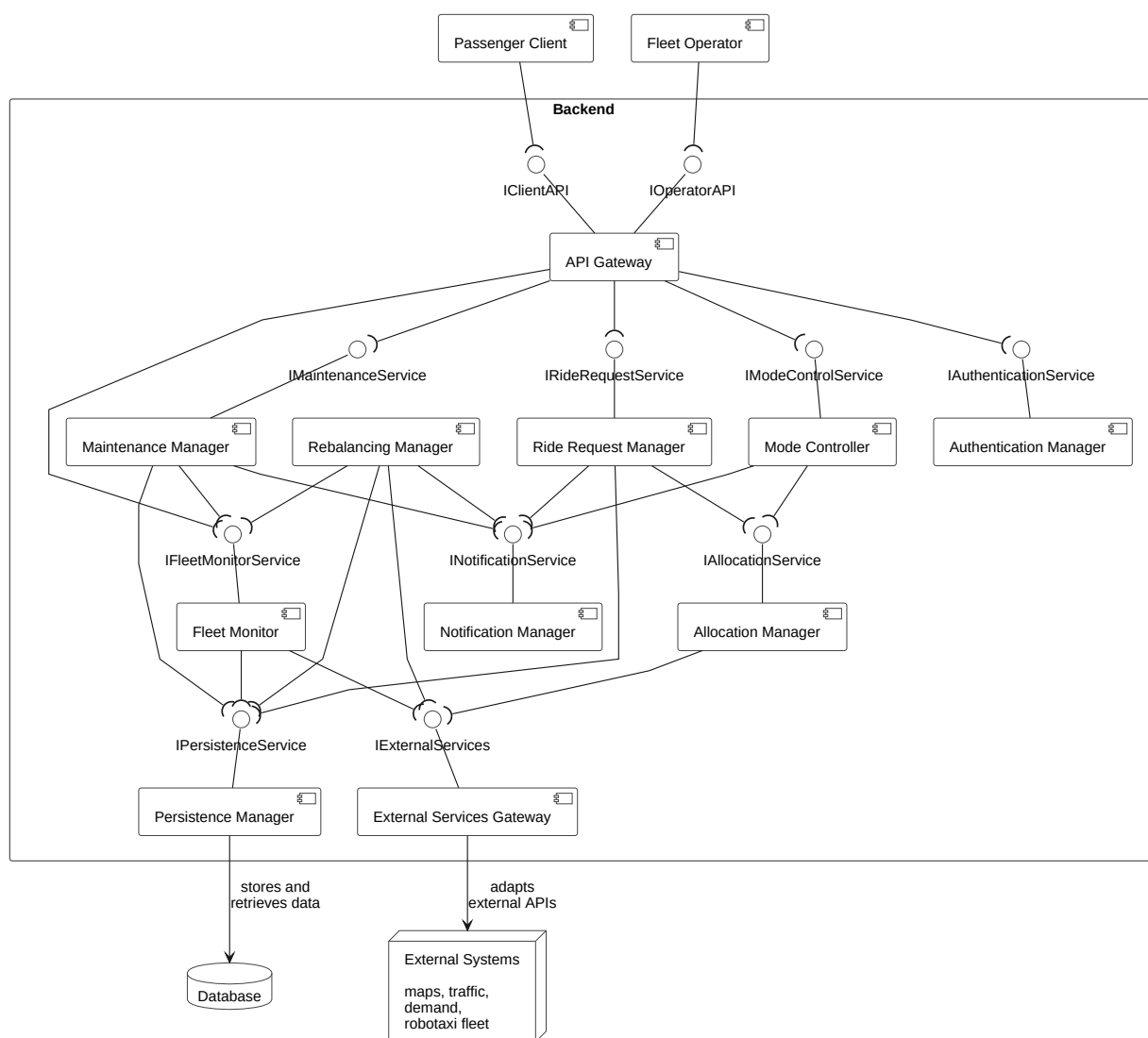


Figure 2.1: High-level component diagram of ROAd, with the two clients, the backend managers, the database, and the external systems

The main components and the operations they export are:

- ▶ **API Gateway:** Single entry point for the clients. Exposes the REST endpoints and the Web-Socket channel used to push notifications. Routes each call to the right manager and enforces authentication.
- ▶ **AuthenticationManager:** `authenticate()`, `register()`, `updateProfile()`. Manages accounts and sessions (R1, R2).
- ▶ **RideRequestManager:** `submitImmediate()`, `submitAdvance()`, `cancel()`. Validates requests, coordinates advance reservations and drives the request lifecycle (R3, R4).
- ▶ **AllocationManager:** `allocate(request,candidates)`, `setActiveStrategy()`. Selects the robotaxi to assign by delegating to the active *allocation strategy* (R5, R8). This is the *context* of the Strategy pattern.
- ▶ **ModeController:** `onTrafficLevel()`, `setManual()`, `enableAuto()`. Implements Auto/Manual mode, the automatic strategy switching with hysteresis and the priority of human intervention (R12, R13, NFR9, NFR10).

- ▶ **FleetMonitor:** `getCandidates()`, `getAvailableRobotaxis()`, `getFleetStatus()`, `assign()`, `requestRebalancing()`. Keeps the real-time picture of every robotaxi (position, state) and coordinates vehicle lifecycle updates (R7, G6, G8). Each Robotaxi manages its own lifecycle through the *State* pattern.
- ▶ **RebalancingManager:** `analyzeDemand()`, `rebalance()`.
Identifies high-demand zones and repositions available robotaxis (R10, R11, G9).
- ▶ **MaintenanceManager:** `requestMaintenance()`, `completeMaintenance()`.
Marks robotaxis in/out of maintenance and prevents their assignment (R9).
- ▶ **NotificationManager:** `update(event)`.
Receives domain events and dispatches notifications to the interested clients (R6, G7). Concrete *observer* of the Observer pattern.
- ▶ **PersistenceManager:** `create()`, `update()`, `filterAvailable()`, `reserve()`. The only component that talks to the database; it filters candidates according to their availability timelines and atomically stores each advance booking together with the corresponding robotaxi reservation. It is where the uniqueness/consistency constraints live (NFR4, C1).
- ▶ **ExternalServicesGateway:** `getETA()`, `getTraffic()`, `getDemandData()`, `commandRoute()`, `readTelemetry()`.
A facade over the external systems (mapping service, demand data source, robotaxi fleet), with one adapter per provider (NFR8).

2.3 Class view

This section zooms into the three components that carry the patterns required for the project. The remaining components are mostly procedural and do not need a class-level view.

2.3.1 AllocationManager (Strategy)

Figure 2.2 shows the classes of the allocation logic. The `AllocationManager` holds a reference to an abstract `AllocationStrategy` and calls `selectRobotaxi()` on it; the two concrete strategies (`NearestAvailableStrategy`, `MinimumETAStrategy`) implement the actual policy. The `ModeController` is the object that swaps the active strategy at runtime.

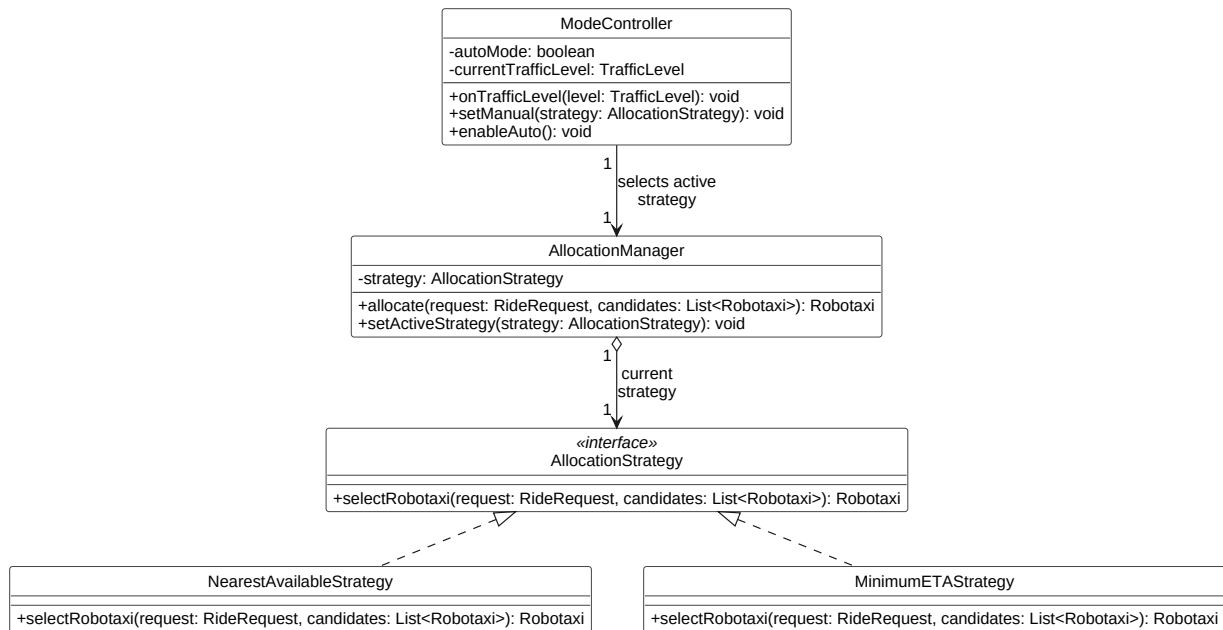


Figure 2.2: Class diagram of the AllocationManager: the Strategy pattern applied to vehicle allocation

2.3.2 Robotaxi (State)

Figure 2.3 shows the lifecycle classes. The **Robotaxi** keeps a reference to the **RobotaxiState** interface. The concrete states are **AvailableState**, **AssignedState**, **ArrivingState**, **ArrivedState**, **InRideState**, **RebalancingState** and **MaintenanceState**. Each concrete state defines the behaviour of the transitions that are legal from it, while calls for any other transition are rejected. This makes illegal transitions impossible by construction (NFR5).

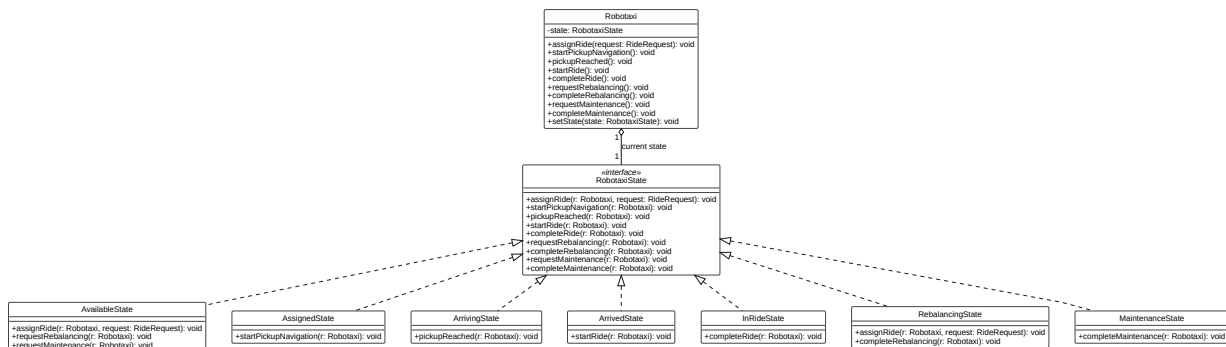


Figure 2.3: Class diagram of the Robotaxi: the State pattern applied to the vehicle lifecycle

2.3.3 NotificationManager (Observer)

Figure 2.4 shows the notification classes. **Robotaxi** and **Ride** are *subjects*, while the **NotificationManager** is the *observer* that is notified whenever a subject changes state. It then dispatches the received events to the **PassengerAppSession** and the **OperatorDashboardSession**.

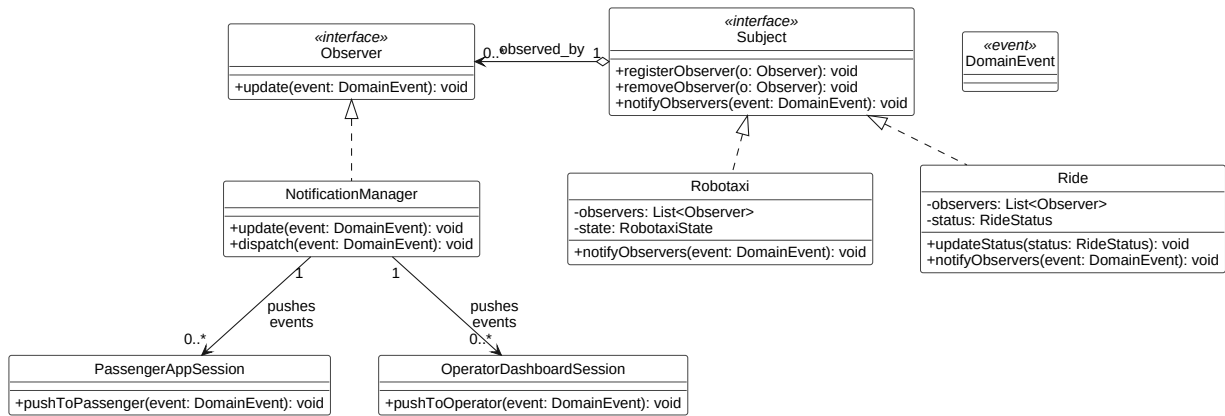


Figure 2.4: Class diagram of the notification mechanism: the Observer pattern used to propagate domain events from subjects to observers

2.4 Runtime view

The sequence diagrams below refine, at component level, the requirement-level scenarios of the RASD. They show the order of messages exchanged among components to accomplish the most relevant tasks.

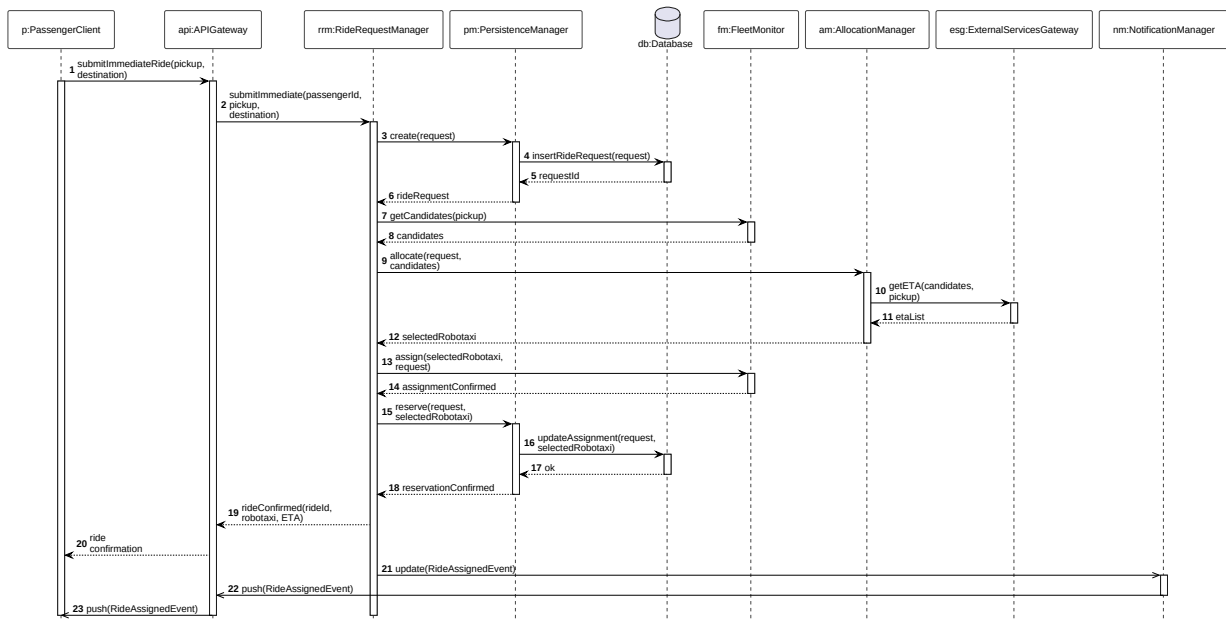


Figure 2.5: Sequence diagram of the immediate ride request and allocation flow, including request persistence, robotaxi selection, assignment, and passenger notification (R3, R5, R6)

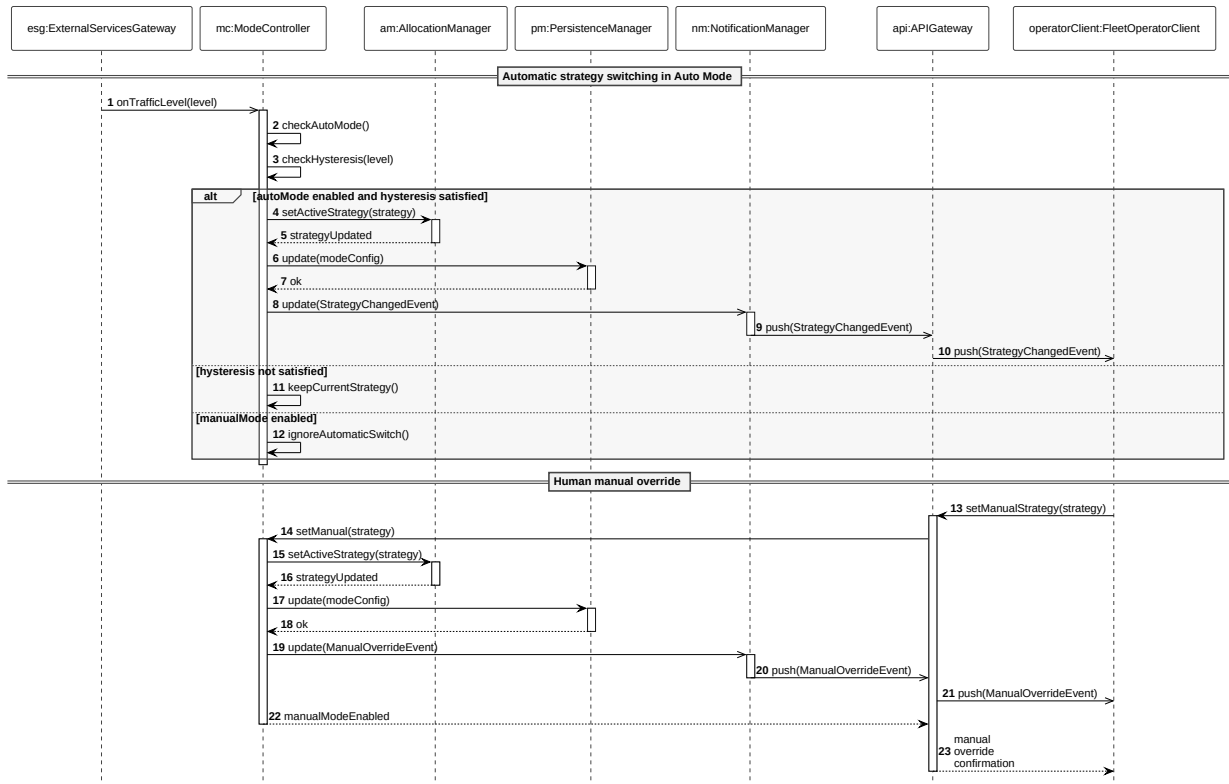


Figure 2.6: Sequence diagram of the automatic strategy switching flow, showing hysteresis-based strategy selection in Auto Mode and the priority of human manual override (R12, R13, NFR9, NFR10)

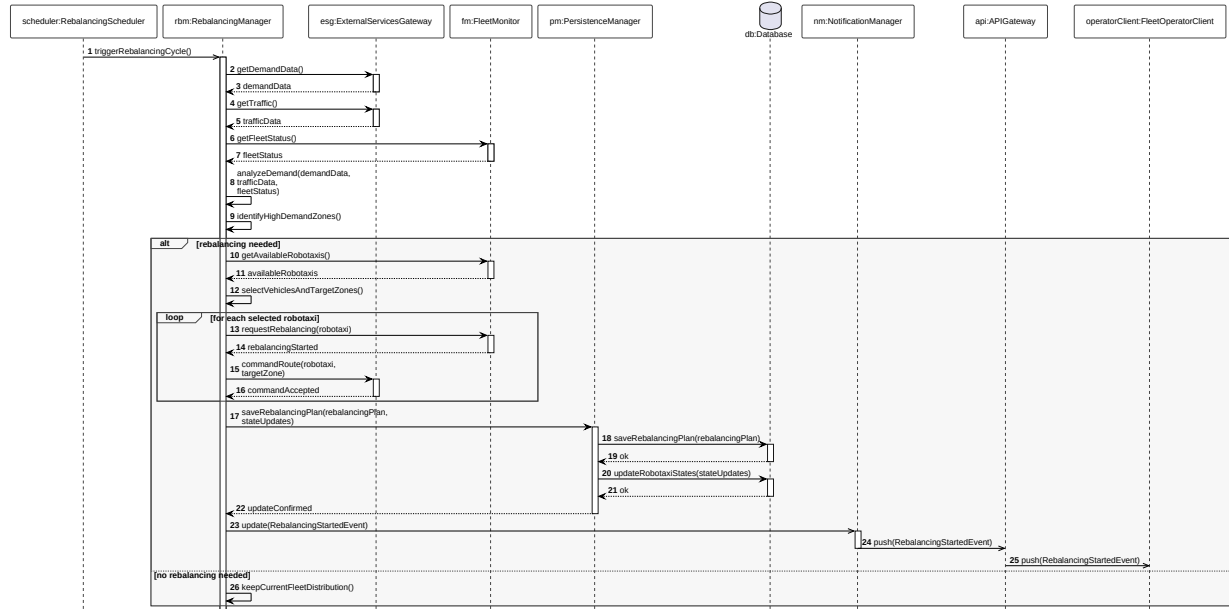


Figure 2.7: Sequence diagram of the dynamic fleet rebalancing flow, including demand analysis, available vehicle selection, and route command dispatch (R10, R11)

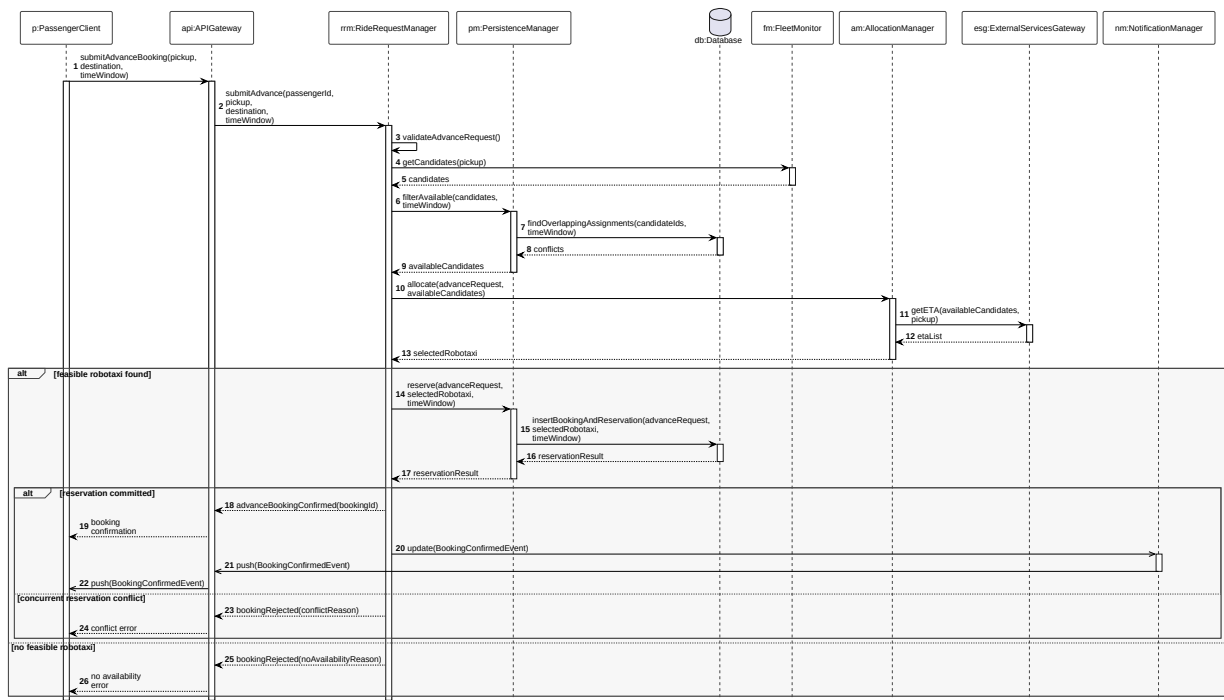


Figure 2.8: Sequence diagram of the advance booking flow, including request validation, future vehicle allocation, conflict prevention, atomic booking and robotaxi reservation, and passenger notification (R4, NFR4, C1).

2.5 Deployment view

Figure 2.9 shows how the components are deployed onto physical nodes. The clients run on the users' devices, the backend on an application server (replicable), the database on its own node, and the robotaxis expose an on-board control API reached over the network. The `PersistenceManager` is the only component that accesses the database, while communication with the robotaxis and the other external systems is handled by the `ExternalServicesGateway`.

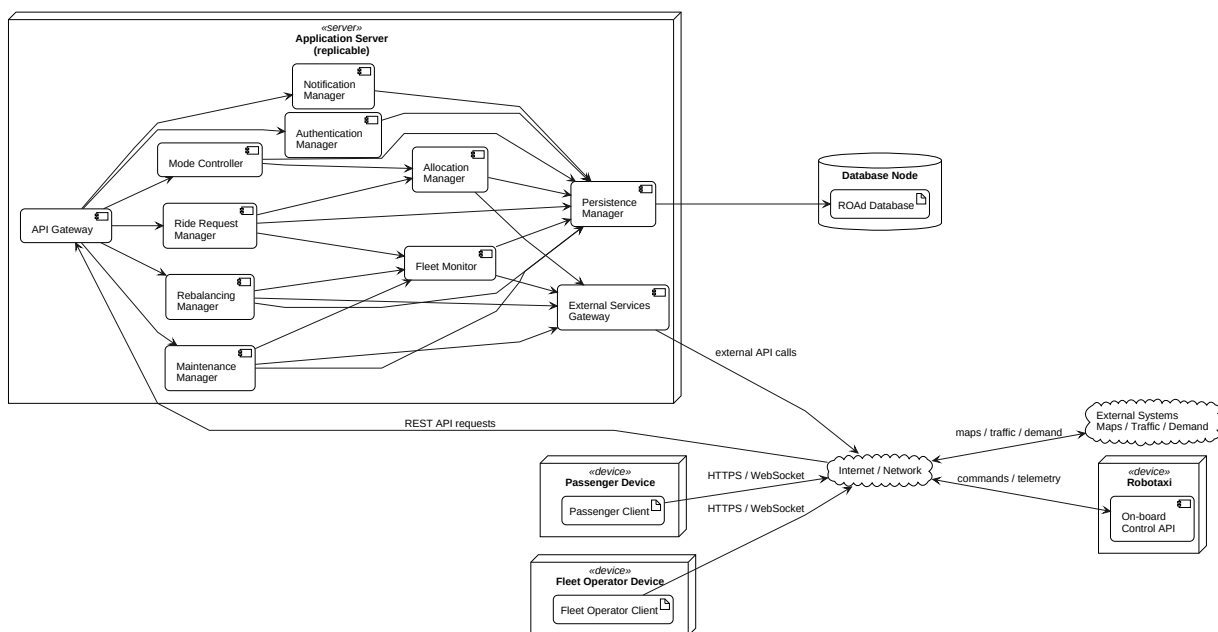


Figure 2.9: Deployment view of the three-tier architecture.

2.6 Selected architectural styles and patterns

We document here the design patterns we adopt. For each one we follow the same scheme used in the course: the *problem*, the *solution* in our system, and the *reason* why the pattern is the right answer. The first three are the patterns explicitly requested for this project.

2.6.1 Strategy — allocation algorithms

Problem. The system must assign a robotaxi to a request, but there is no single “best” rule: depending on traffic, the operator may want the nearest available vehicle (to save distance) or the vehicle with the minimum ETA (to cut waiting time during congestion). The rule must be interchangeable at runtime, and new rules should be addable later without touching the request-handling code (NFR7).

Solution. We define an abstract `AllocationStrategy` with a single operation `selectRobotaxi(request, candidates)`, and two concrete subclasses (`NearestAvailableStrategy`, `MinimumETAStrategy`). The `AllocationManager` is the *context*: it owns a reference to the abstract strategy and never to a concrete one. The `ModeController` replaces the active strategy via `setActiveStrategy()` (see Figures 2.2 and 2.6).

Why. Strategy turns “which algorithm” into a runtime choice instead of a hard-coded branch, which is exactly what R8/R12 ask for. It also makes each policy independently testable and keeps the allocation logic open for extension but closed for modification.

2.6.2 Observer — state-change notifications

Problem. Several parties are interested in the same events — when a robotaxi changes state or a ride progresses, the passenger app must update, the operator dashboard must refresh, and the notification logic must fire. The producers of the events should not need to know who is listening, and listeners may come and go (a passenger opens/closes the app).

Solution. `Robotaxi` and `Ride` implement the `Subject` interface, keep a list of `Observers` and call `notifyObservers(event)` on every state change. The `NotificationManager` implements `Observer.update(event)` and dispatches each received event to the `PassengerAppSession` and the `OperatorDashboardSession`. The `APIGateway` forwards the corresponding push messages to the connected clients through its `WebSocket` channel (Figure 2.4).

Why. Observer realises the one-to-many dependency between a subject and its listeners with minimal coupling, and supports dynamic subscription. It is the natural way to satisfy R6/G7 (real-time updates) and, over a `WebSocket` push channel, also helps NFR2 (near-real-time responsiveness) by avoiding polling.

2.6.3 State — robotaxi lifecycle

Problem. A robotaxi behaves differently depending on its state (available, assigned, arriving, arrived, in ride, maintenance, and, added in this design, rebalancing), and only some transitions are legal. Encoding this as a large conditional spread across the code is error-prone and risks reaching invalid states (NFR5).

Solution. The `Robotaxi` (context) delegates its behaviour to a `RobotaxiState` object. Each concrete state defines the behaviour of the events that are legal from it; any other call raises an `InvalidTransition` error. The full state machine is shown in Figure 2.10; the classes in Figure 2.3.



Figure 2.10: Design-level state machine of the Robotaxi, realised by the State pattern. The *Rebalancing* state is added with respect to the RASD FSM.

Why. State localises each transition rule next to the state it belongs to, so illegal transitions become impossible by construction. It directly supports G6 (lifecycle tracking) and NFR5 (robustness), and makes the (finite) set of transitions easy to test exhaustively.

2.6.4 Other patterns and styles

- ▶ **Adapter / Facade** — the `ExternalServicesGateway` exposes a uniform interface to the mapping service, the demand source and the vehicle API, hiding their specific protocols (NFR8).
- ▶ **MVC** — both clients separate the observable model (a local copy of the relevant server state) from the view and the controller, so the view is a function of the model.
- ▶ **Singleton** — the `FleetMonitor` is a single dependency-injected instance within each backend process. Its in-memory fleet index is a disposable cache synchronised with persistent data and external telemetry, and is not the authoritative shared state.

User Interface Design

This chapter gives an overview of the two user interfaces through wireframes. They are low-fidelity on purpose: the goal is to show structure and the main interactions, not the final graphic design. Both interfaces are designed to be intuitive and require no training (NFR6).

3.1 Passenger mobile app

The passenger app is centered on a map. The main screen (Figure 3.1) lets the user set pickup and destination, choose between an immediate ride and a scheduled one, and request the ride with a single button. After the request, the same screen turns into a live status view that follows the ride through its states (searching, assigned, arriving, arrived, in ride), driven by the Observer notifications.

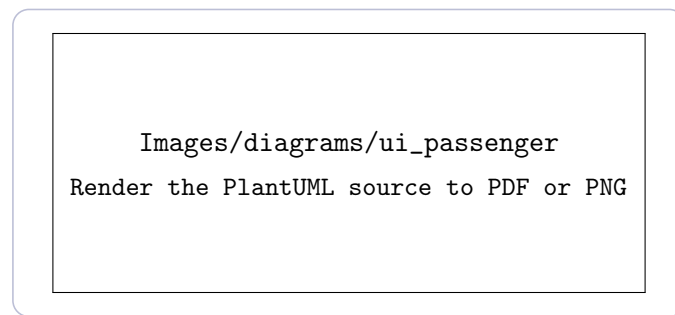


Figure 3.1: Wireframe of the passenger app: request a ride and follow its status.

3.2 Operator web dashboard

The dashboard (Figure 3.2) is a command-and-control console. It shows the live fleet on a map, a strategy panel where the operator sees the active allocation strategy and can switch it (moving the system to Manual mode), and an alerts panel that surfaces automatic strategy switches and rebalancing suggestions. A status bar summarises the fleet by state. The Auto/Manual toggle makes the current control mode always visible (NFR10).

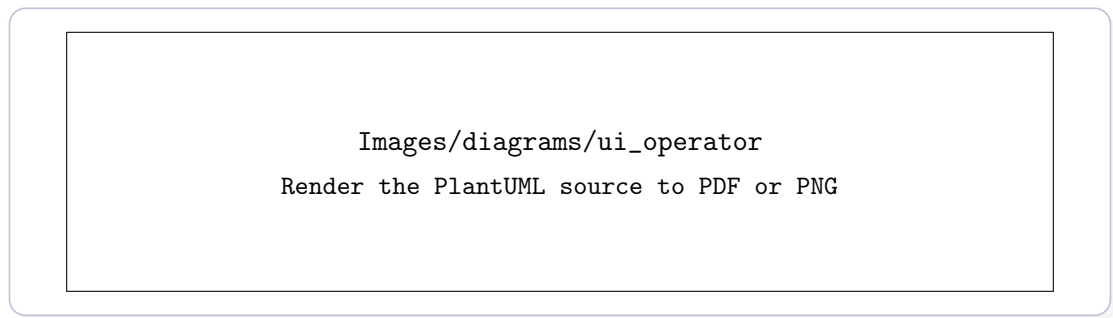


Figure 3.2: Wireframe of the operator dashboard: fleet monitoring and strategy control.

Requirements Traceability

This chapter shows how the requirements defined in the RASD are covered by the design elements of this document. Each functional requirement is mapped to the components (and, where relevant, the patterns) that realise it; each non-functional requirement is mapped to the design decision that supports it.

4.1 Functional requirements

Req	Description (short)	Components / patterns
R1	Registration and login	AuthenticationManager; clients
R2	Profile management	AuthenticationManager
R3	Immediate ride request	RideRequestManager; AllocationManager; Passenger App
R4	Advance booking (no conflicts)	RideRequestManager; FleetMonitor; AllocationManager + <i>Strategy</i> ; PersistenceManager (filterAvailable/reserve)
R5	Automated vehicle allocation	AllocationManager + <i>Strategy</i> ; FleetMonitor
R6	Ride status notifications	NotificationManager + <i>Observer</i> ; API Gateway (push)
R7	Real-time fleet monitoring	FleetMonitor; Operator Dashboard
R8	Runtime strategy selection	ModeController; AllocationManager + <i>Strategy</i>
R9	Maintenance management	MaintenanceManager; FleetMonitor + <i>State</i>
R10	Demand analysis	RebalancingManager; ExternalServicesGateway
R11	Proactive fleet rebalancing	RebalancingManager; FleetMonitor
R12	Auto Mode (traffic-driven switching)	ModeController; ExternalServicesGateway
R13	Manual Mode (override)	ModeController; Operator Dashboard

4.2 Non-functional requirements

NFR	Quality	Design decision that supports it
NFR1	Concurrency handling	Replicable application tier behind API Gateway; DB transactions in PersistenceManager
NFR2	Real-time responsiveness	Observer + WebSocket push instead of polling
NFR3	Scalability	Horizontally replicable backend; single data tier
NFR4	Data consistency	Uniqueness/timeline constraints in PersistenceManager; atomic <code>reserve()</code>
NFR5	Robustness (valid transitions)	State pattern in Robotaxi; FleetMonitor coordinates lifecycle updates
NFR6	Ease of use	Map-centric app and command-and-control dashboard (Ch. 3)
NFR7	Extensibility of allocation logic	Strategy pattern; AllocationManager closed for modification
NFR8	Separation of concerns	ExternalServicesGateway (Adapter/Facade); layered backend
NFR9	Stability of auto-switching	Hysteresis in ModeController (revert only at LOW)
NFR10	Priority of human intervention	Manual mode in ModeController suspends auto switches

Implementation, Integration and Test Plan

5.1 Feature prioritisation

We rank the features by importance (value for the core service) and difficulty, to decide what to build first. The viable prototype must cover the high-importance features; the rest are nice-to-have.

Feature	Importance	Difficulty
Accounts and authentication (R1, R2)	High	Low
Immediate ride + allocation (R3, R5)	High	Medium
Robotaxi lifecycle / State (R9, G6)	High	Medium
Conflict-free advance booking (R4)	High	High
Notifications / Observer (R6)	High	Medium
Fleet monitoring dashboard (R7)	High	Medium
Runtime strategy selection (R8)	Medium	Low
Auto mode + hysteresis (R12, NFR9)	Medium	High
Demand analysis + rebalancing (R10, R11)	Medium	High
Manual override (R13, NFR10)	Medium	Low

5.2 Implementation and integration order

We integrate **bottom-up**, so that every component is integrated only after the components it depends on are already in place and tested. External systems are replaced by *stubs/mocks* at the beginning and connected through the `ExternalServicesGateway` later.

1. **PersistenceManager + database schema:** foundation for everything; includes the uniqueness/timeline constraints (NFR4).
2. **FleetMonitor + Robotaxi (State):** the vehicle model and its lifecycle, first against a vehicle *mock*.
3. **AllocationManager + strategies (Strategy):** on top of FleetMonitor.
4. **RideRequestManager:** immediate first, then advance booking with candidate filtering, vehicle allocation and atomic reservation.
5. **NotificationManager (Observer) + API Gateway push:** wire events to clients.
6. **ModeController + RebalancingManager:** the higher-level control logic.

7. **ExternalServicesGateway adapters:** replace the mocks with the real mapping and demand services.
8. **Clients:** Passenger App and Operator Dashboard against the stable API.

5.3 Testing strategy

We follow the **V-model**: each artifact is verified against the specification it refines. Verification (“are we building the software right?”) is our focus; validation is addressed through the demos with the teacher.

Unit testing

Each manager and each pure-logic class in isolation. Highest priority: (i) the two `AllocationStrategy` subclasses, with equivalence partitioning over the candidate set (empty fleet / no suitable vehicle / one suitable / several) and boundary values on ETA and distance; (ii) the `RobotaxiState` transitions, tested *exhaustively* because the state machine is finite, including the rejection of every illegal transition (NFR5); (iii) the conflict check of advance booking (NFR4).

Integration testing

The request→allocation→notification chain, and the booking→reserve→persistence chain, with external systems mocked. We test the integration in the same bottom-up order used to build it.

System testing

The four RASD scenarios end-to-end on a staging deployment: immediate ride, advance booking, auto/manual strategy management, dynamic rebalancing. Includes a concurrency test that fires two requests for the last available robotaxi and checks that exactly one is assigned (NFR1, NFR4).

Acceptance testing

Walkthrough of the goals (G1–G10) with the teacher at the milestone demo.

Tools. A unit-test framework for the chosen backend language (e.g. JUnit or pytest), an HTTP client for API/integration tests (e.g. Postman/newman or REST-assured), and a simple load generator for the concurrency test. The repository runs the test suite in continuous integration on every push, as required by the project rules.

References

- ▶ ROAd: Robotaxi Optimized Allocation — Requirements Analysis and Specification Document, v1.0.
- ▶ M. Camilli, *Software Engineering for Automation — Course Slides*, Politecnico di Milano, A.Y. 2025–2026 (in particular: modular design, OO design, design patterns, V&V).
- ▶ E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- ▶ M. Jackson and P. Zave, *The World and the Machine*, ICSE, 1995.
- ▶ Object Management Group, *Unified Modeling Language Specification*, <https://www.omg.org/spec/UML/>.
- ▶ PlantUML, <https://plantuml.com> (diagram sources in ./Images/diagrams/).